Quality Code 0.68

The commands, and what is new. From Jira ticket to tests you can trust.

What is new since 0.44

A shorter spec (0.68). /quality-code:spec drops --gate-only (use ticket --eval) and --yes (use --ci); its hint shows only [KEY] [--dry-run] [--ci].

Copilot loads all the commands (0.67.1-0.67.2). In GitHub Copilot CLI the commands were not found, then only four showed. The plugin now has a Copilot manifest (.plugin/plugin.json), and every command header is valid YAML. In Copilot: copilot plugin update quality-code@teamsystem, then restart copilot. Claude Code is unchanged.

Help for your role (0.67). /quality-code:help says whether setup is done, then what you can do now for your role, with how each command is used. --dev, --qa, --po, --all, or a command name.

Update is one word (0.66). check-update is now /quality-code:update: the same version check, and the update on your yes.

One check command (0.65). run-pipeline, run-backend, run-frontend and run-smoke are now /quality-code:check: no flag for the full suite (--branch, --evals), --backend or --frontend for one side, --smoke [--url URL] for the @smoke tests.

Clean the cache with clean (0.64). clean-cache is now /quality-code:clean --cache: the same list, question and delete; --all for every project, --yes skips the question. The full clean still never skips its yes.

One report command (0.63). kpi and consumption are now flags of report: /quality-code:report --kpi (the default) shows the KPIs, /quality-code:report --cost the AI usage and estimated cost. Every other flag works as before.

One config command (0.62). config-get and config-set are gone: /quality-code:config get <key>, config set <key> <value> and config keys do the same, with the same checks.

Windows paths (0.61.2). On Windows, a Vitest spec whose tests ran is no longer reported as never ran, and a node --test spec that does not load is reported as never ran, not as a failed test.

How to use it, by role (0.61.1). Setup ends with a fixed block: one line each for developers (spec, run-pipeline), QA (ticket --eval, xray, qa) and PO (ticket <PROJECT> --eval, transcript-goal). A Jira-only setup leaves out the developers line.

One ticket command (0.61). ticket-eval and tickets-eval are now flags of ticket: /quality-code:ticket <KEY> --eval scores one ticket, /quality-code:ticket <PROJECT> --eval the latest 50 of a project. The --json, --refuse-comment, --limit and --all flags are gone.

The guide, from update (0.60.2). /quality-code:update ends its card with the path of this guide, shipped with the plugin (assets/commands.pdf): every command, and what is new in the installed version.

Fixes from the 5 Nov feedback (0.60.1). A title "FE -", "FE-" or "FE:" is read as frontend (also BE, FS), so the side is not asked. An epic is refused ("pass one of its stories or tasks"). Placeholder criteria (lorem ipsum, TBD, N/A) no longer pass the score. A run says once when screenshots are not read because the Jira files key is not set. Every Jira write ends with "Co-authored via Quality Code".

A sign-in page on the shared server (0.60). The shared MCP server shows a Quality Code page at its address: Sign in with Microsoft, then the /mcp address and the commands to connect Claude Code, Copilot CLI or Cursor. Needs a Web redirect URI and QUALITY_CODE_ENTRA_CLIENT_SECRET; /mcp itself is unchanged. Nothing changes in the commands.

Config lives on your machine (0.59). Setup no longer writes into your repo. config.json is kept next to the tickets and run logs, in the place the last setup question picks. A repo with .claude/quality-code.json still works; CI jobs still read the repo file.

QA runs in the pane you watch (0.58). qa takes the test execution key (qa <KEY>), runs test by test in the browser pane, and you sign in yourself per role. A task key gives the subtasks, merge requests and an end-to-end plan; nothing goes to Jira without a preview and your yes.

xray reads the code and needs no repo (0.57-0.58). Works with the personal setup, adds cases from the GitLab or GitHub links of the story, and ends with Next: /quality-code:qa. Rules come from the hosted Central Context hub by default.

Tests in your flow and ours (0.56). testFlow both writes each test in the project's own flow and again in Quality Code's, so the pipeline can still trace the ticket.

ticket-eval, run and consumption (0.58). ticket-eval takes the key from the branch; run commands offer to fix compile errors; consumption --session shows this session only; spec lets you pick several code files.

Setup only connects (0.55). Setup no longer offers to add Playwright or Vitest; it ends with Setup completed and a table of what is connected. spec offers the runner card (Add it / Not now) at the plan stage when the repo lacks the runner.

config-set and config-get (0.55). Their own commands; tokens are never set there and show the /plugin configure steps instead.

Richer cards and report pages (0.54). Result cards get a headline row (verdict, score, bar with the gate, counts), a Next line and collapsible detail. kpi, tickets-eval and consumption take --report and write one self-contained HTML page with charts. The newer commands follow the same layout.

Clear the cache, or undo the setup (0.53). clean-cache deletes only the local cache. clean removes what the plugin created (setup files, saved output, cache; your tests and personal config only if you pick them) and always asks one explicit yes.

config set (0.53). Change one setting directly, for example config set git.base develop. Checked by the setup rules; nothing is written on an error or a secret.

More kinds of tests (0.52). Negative cases on every ticket, an axe accessibility scan, optional visual regression, database checks with Testcontainers, a regression test from a Bug, and a smoke suite (run-smoke, ci --smoke).

Sprint goal from a Teams transcript (0.52). transcript-goal turns a planning transcript into a sprint goal and tasks, with review cards; Jira only after a preview and your yes.

Rewrite a weak story (0.51). After ticket --eval, a script flags a long or thin description; rewrite it from the description, Figma or Confluence, then post to Jira or save locally.

Memory, language and pickers (0.50). Commands remember where you stopped and what they learned about the project; your language is kept; every question is a clickable picker; new tests go into the existing test structure first and follow the project's conventions before the stack's defaults.

Plan card (0.48). Before any test is written, spec shows what will be written and asks Yes, No or Something else, using your own libraries and flow when you have them. After spec it suggests run-backend or run-frontend.

Update first

```
/plugin update quality-code@teamsystem
/reload-plugins
```

Type /quality-code: and check that the list shows check, clean, report and transcript-goal. Or run `/quality-code:update`.

1. Set up and settings

Command	What it does	Example
`/quality-code:setup`	Checks the connectors, finds the project, its stack, Jira project and base branch, shows one card to confirm and writes .claude/quality-code.json. Option 2 is a personal setup for QA and PO with no repo.	`/quality-code:setup` `/quality-code:setup advanced`
`/quality-code:config`	Shows a short summary and changes one setting at a time. Directly: get , set (dotted keys, lists with commas; --personal, --project, --yes) and keys. Tokens and plugin settings (Jira email, GitLab, Xray) are checked, never echoed.	`/quality-code:config` `/quality-code:config set git.base develop` `/quality-code:config get jira.site`
`/quality-code:help`	Says whether setup is done, then the steps for your role with how each command is used. --dev, --qa, --po another role, --all every command, a name one command.	`/quality-code:help --all`
`/quality-code:update`	Says whether a newer version is out and what changed; offers to update.	`/quality-code:update`
`/quality-code:clean --cache`	Deletes only the plugin's local cache (saved stops, reused facts, language) for this project, or every project with --all.	`/quality-code:clean --cache --all`
`/quality-code:clean`	Removes what the plugin created in the project. A script lists it by group; you pick the groups and say yes once. Never skipped by --yes or --ci. --dry-run only lists.	`/quality-code:clean --dry-run`

2. Every day: from ticket to tests

Command	What it does	Example
`/quality-code:spec`	The main command. Runs the readiness gate, shows the plan (with Negative, Accessibility, Visual and Database lines when they apply), writes the tests in your test folders, runs the pipeline and reads the card back. A Bug ticket leaves a @regression test.	`/quality-code:spec ABC-123` `/quality-code:spec ABC-123 --dry-run`
`/quality-code:ticket --eval`	Scores one ticket out of 100 (70 passes). If the story is long or thin, offers to rewrite it from the description, Figma or Confluence; post to Jira after a preview, or save locally.	`/quality-code:ticket ABC-123 --eval`
`/quality-code:ticket`	Finds a ticket and shows it in short, with tips for its author. Read-only.	`/quality-code:ticket assignee:me`
`/quality-code:transcript-goal`	Reads a Teams planning transcript (file, pasted text or the Microsoft 365 connector), drafts the sprint goal and tasks, asks only what was not said, shows review cards, then creates the tasks in Jira after a preview and your yes, or saves them locally.	`/quality-code:transcript-goal --file planning.vtt`

3. Run and grade the tests

Command	What it does	Example
`/quality-code:check`	The full suite. --branch: only the tests your branch changed. --evals: the frozen evals.	`/quality-code:check --branch`
`/quality-code:check --frontend`	Every Playwright spec of the project.	`/quality-code:check --frontend`
`/quality-code:check --backend`	Every backend spec: Vitest, pytest, JUnit, Go or .NET.	`/quality-code:check --backend`
`/quality-code:check --smoke`	Only the tests tagged @smoke, against the running app or any URL. Read-only. Frontend only for now.	`/quality-code:check --smoke --url https://staging.example.com`

The card ends in one verdict, decided by a script: green, tests red, checks failed or not verified. A Bug ticket also shows the fail-then-pass proof (--regression-before red|green).

4. CI and scheduled checks

Command	What it does	Example
`/quality-code:ci`	Writes the CI file for GitHub Actions or GitLab: a quick quality-gate and a quality-tests job. --smoke adds an optional post-deploy job that runs the smoke suite against SMOKE_URL.	`/quality-code:ci github` `/quality-code:ci --smoke`
`/quality-code:routine`	Schedules a run with Claude's scheduler; reports a change from green to red; --stability N finds flaky tests; --regression runs the @regression suite.	`/quality-code:routine run --regression`

5. For QA and product owners

Command	What it does	Example
`/quality-code:ticket --eval`	Scores a whole Jira project, read-only. --report writes an HTML page with charts.	`/quality-code:ticket ABC --eval`
`/quality-code:xray`	Writes Xray test cases from a ticket as a CSV and case.xlsx.	`/quality-code:xray ABC-123`
`/quality-code:qa`	Runs your Xray manual tests with an agent; you give every verdict; nothing reaches Jira without a preview.	`/quality-code:qa --resume`
`/quality-code:seed`	Loads realistic mock data for a ticket into your local database only.	`/quality-code:seed ABC-123`

6. Numbers

Command	What it does	Example
`/quality-code:report --kpi`	Tickets gated, fill-ins, verdicts and the Quality Program numbers. Read-only. --report writes an HTML page with charts.	`/quality-code:report --kpi --since 30d --report`
`/quality-code:report --cost`	AI usage and estimated cost of the plugin's runs, read locally. --report writes an HTML page with charts.	`/quality-code:report --cost --by command`